

Client-Centric Consistency in a Three-Node Cassandra Cluster

Jia Siqi Jia Zeyue

DSA5208 Project 1, National University of Singapore

Date: 24 September 2026

1 Question and scope

We use a trajectory-based runner [5] to investigate how Cassandra’s consistency settings change what a client sees when it moves between nodes. The four properties are read-your-writes (RYW), monotonic reads (MR), monotonic writes (MW), and writes-follow-reads (WFR). RYW concerns a client’s earlier writes; MR concerns its earlier reads. MW orders a client’s successive writes, while WFR relates a write to the updates previously read by its writer [1].

Our dataset contains **316 completed experiments**, covering all 36 combinations of four properties, three consistency settings and three scenarios. Each combination has eight or nine observations. The initial plan was larger, but execution was stopped after repeated preparation failures. We retain all completed experiments, including unexpected outcomes. These histories form the dataset analysed in this report.

RYW and MR are checked directly from successful client operations. MW and WFR use immutable dependency markers and a local-replica probe. These probes test predecessor visibility, not every internal application event. Their scope matters when interpreting a result labelled VIOLATION.

2 Deployment and configuration

Three Cassandra 5.0.9 containers, N1–N3, share the Docker bridge `project1-net` and datacenter `dc1`. The keyspace uses `SimpleStrategy` with replication factor 3, so every key has a replica on each node. A fourth container runs the Python workload and fault controller. Separate driver sessions pin requests to the selected coordinator. This identifies the coordinator, not necessarily the replica serving its read.

The client environment records Python 3.11.16, `cassandra-driver` 3.30.1, `docker-py` 7.2.0 and Docker Engine 29.5.2. The dedicated Colima VM was configured with four CPU cores and 8 GB RAM on an Apple Silicon host. Each Cassandra container has a 2 GB limit, a 768 MB Java heap and 16 tokens. The supplied Dockerfiles pin the base image digests and Python dependencies; per-case `environment.json` files retain runtime versions, addresses and image IDs.

Installation builds the image with `iptables`, starts the nodes in health-check order, and creates the experimental keyspace and table. The table stores an item key, version, write ID, client and sequence number. The main dataset uses one keyspace with fresh UUID-based keys for every case. Sequence numbers encode intended order; they are not evidence of replica execution order.

At RF=3, ONE, QUORUM and ALL wait for one, two and three replica responses respectively. Writes are sent to all replicas regardless of the acknowledgement threshold. A successful quorum write and quorum read intersect in at least one replica [2]. We test matched read/write levels; setup always uses ALL. The table uses `read_repair='BLOCKING'` and disables speculative retry. Blocking read repair is relevant to monotonic quorum reads [3]. The driver has a three-second timeout and no automatic request retries; server read/write timeouts are 1.5 seconds. We use increasing timestamps and one controlled version change, without competing writers, deletions or TTLs.

3 Predictions and experimental method

3.1 Predictions recorded before execution

The original `plan.json` fixes the case order and predicted classifications. Normal workloads were expected to complete without a counterexample. With N3 stopped, ONE and QUORUM were expected to complete on the live side, whereas ALL would lack a required response. For the N3 | N1/N2 partition, ONE was expected to expose stale reads or missing dependencies; QUORUM and ALL were expected to leave a required operation incomplete. These are predictions for the chosen workloads, not claims that every ONE execution must be inconsistent.

For RYW/MR, quorum intersection motivates the prediction about successful single-key reads. For MW/WFR, we initially expected connected replicas to contain both markers by the time of the probe. This was a timing-dependent expectation: two quorum writes do not require every individual replica to contain both keys. The results below include a case that contradicts this initial expectation.

3.2 One full fault cycle per case

Each case first restores connectivity, checks membership from all three nodes, establishes fresh client sessions and writes `x=0` and `y=0` at ALL. It verifies `x` through each coordinator before starting the workload. These five successful setup requests are retained separately from the tested operations.

The normal scenario injects no fault during the workload. The node-stop scenario stops N3, confirms it is stopped and waits until the live nodes mark it down. This models unavailability, not abrupt power loss. The partition blocks N3's internode TCP ports 7000/7001 in both directions, preserving client access on 9042. The controller saves firewall rules, checks blocked connections and waits for the expected membership views. After observations, it heals the rules, restarts stopped nodes and verifies membership and connectivity. Each case has its own preparation and recovery cycle; the machines and database data directory are shared.

Table 1: Workloads and evidence. The destination/cause node is N3, or N2 when N3 is stopped.

Test	Workload	Counterexample sought
RYW	A writes $x=1$ at N1, then reads x at the destination.	A’s successful read returns 0 after its successful write.
MR	B writes $x=1$ at N1; A reads 1 at N1, then reads at the destination.	A actually obtains 1, then successfully reads 0.
MW	A writes x at the cause node, then writes dependent y at N1.	A verified local N1 probe sees $y=1$ but $x=0$.
WFR	B writes x ; A reads that exact write, then writes y at N1.	The same local gap, with A’s dependency justified by its actual read ID.

For MW/WFR, immediately after that case’s successful workload, N1 is isolated for measurement. After verifying isolation, the observer reads y and then x at ONE through N1. Successful reads can then use only N1’s local replica. The markers are immutable. The additional isolation is a measurement intervention, including in a case labelled normal; it is recorded separately from the workload fault.

3.3 Sample selection and interruptions

The planned 1,080 cases consisted of 30 shuffled blocks of the 36 combinations, using seed 5208. We report the completed prefix: eight full blocks (288 cases) and the first 28 cases of block nine. Therefore 28 combinations have nine cases and eight combinations have eight.

The first process completed cases 1–93 and stopped during case 94’s preparation when an N3 read timed out. A resumed process retained those files unchanged and completed cases 94–316. It added a three-second preparation wait and allowed up to three fresh-key preparation attempts, with health checks and a ten-second wait between failures. Formal workload operations were never retried. At case 317, all three preparation attempts failed: an ALL write received two of three required responses. No formal workload ran in either interrupted attempt. Both are preserved outside the 316-case denominator.

We stopped collection at this point and kept every completed case, regardless of result. The cutoff was chosen after interruptions, not fixed in advance. The two phases also have different preparation policies. These facts prevent treating the sample as a balanced set of 30 repetitions or as independent random samples from a production failure distribution.

4 Results and discussion

4.1 Results across the 36 combinations

Replaying all 316 histories reproduced **172 PASS**, **38 VIOLATION** and **106 INCONCLUSIVE** classifications. PASS means the necessary observations completed without the selected counterexample. INCONCLUSIVE means the history lacks a required successful operation; it is not evidence that the property holds. For MW/WFR, VIOLATION refers specifically to the tested local dependency-visibility condition.

Table 2: All selected results. P, V and I denote PASS, VIOLATION and INCONCLUSIVE; numbers are case counts.

Scenario	Workload CL	RYW	MR	MW	WFR
Normal	ONE	P9	P9	P9	P8, V1
Normal	QUORUM	P9	P8	P8, V1	P9
Normal	ALL	P8, I1	P9	P9	P9
N3 stopped	ONE	P8	P8	P9	P9
N3 stopped	QUORUM	P8	P9	P9	P8
N3 stopped	ALL	I8	I9	I8	I9
2+1 partition	ONE	V9	V9	V9	V9
2+1 partition	QUORUM	I8	I9	I9	I9
2+1 partition	ALL	I9	I9	I9	I9

There are 107 normal, 102 node-stop and 107 partition cases. Across them, 658 workload requests were attempted: 552 succeeded and 106 failed. The errors were 105 Unavailable responses and one ReadTimeout. The accepted cases also contain 1,580 successful setup requests and 214 successful measurement reads forming 107 local dependency probes. Failed preparation attempts are retained separately and are not included in these request totals.

The audit checks original file hashes, source snapshots, returned coordinator addresses, fault evidence, probe links and recorded recovery. These checks establish traceability; they do not prove that all four session guarantees hold. Phase 1 contains 50 P, 11 V and 32 I; phase 2 contains 122 P, 27 V and 74 I.

4.2 What the observations show

All nine partition/ONE cases for each property produced the intended witness. In RYW, A successfully wrote $x=1$ at N1 and then read $x=0$ at N3. In MR, A first read 1 at N1 before reading 0 at N3. These are direct backward-visibility observations in the same client session. For MW, A wrote x on the isolated side before writing y on the majority side; the N1 probe then saw $y=1$ and $x=0$. WFR additionally recorded that A had actually read the cause’s write ID before writing y .

The stronger settings changed availability. Under node failure, all 34 ALL cases were inconclusive. Under the partition, all 35 QUORUM and 36 ALL cases were inconclusive. For RYW/MR at QUORUM, a majority-side write could complete, but the isolated-side read could not. MW/WFR could not finish their isolated-side prerequisite. These failed requests explain why the table contains I rather than a consistent or stale read. A timed-out write may still have updated some replicas; we do not assume that a failed acknowledgement means no mutation occurred.

Three cases disagreed with the recorded prediction:

- **Case 308, normal/ALL/RYW:** the write completed but the required read timed out, giving INCONCLUSIVE. No workload fault was injected, so this is an availability exception in the nominal normal scenario.

- **Case 311, normal/QUORUM/MW:** both writes completed at QUORUM, but the later local N1 probe returned $y=1$ and $x=0$. Quorum acknowledgement does not require each individual replica to contain both writes. The evidence concerns a local ONE probe after measurement isolation; it is *not* a counterexample obtained by two QUORUM client reads.
- **Case 314, normal/ONE/WFR:** A read the exact predecessor at N3 and then successfully wrote y at N1. The isolated N1 probe returned $y=1$ and $x=0$. This shows that a completed workload without an injected workload fault can still leave a local dependency gap.

These three outcomes show that connected workloads can produce availability exceptions and local dependency gaps. The server logs near the second interruption show N2 dropping expired read/write messages even when membership checks reported all nodes up. This supports caution about using membership as a readiness check. It does not establish the underlying cause of the timeouts or of either earlier dependency gap.

4.3 Limits of the conclusions

The experiment covers the required properties and scenarios, but its claims are narrow. All nodes share one VM, and each workload is short and sequential. Normal means no new workload fault, not a freshly created cluster: earlier failures may affect subsequent cases. Unique keys separate the logical data, but do not remove shared queues, timing effects or recovery history. The two preparation policies and the interruption-based stopping rule also limit comparisons of frequencies.

The MW/WFR probe checks a necessary predecessor-visibility condition under controlled immutable markers. It neither logs internal write application times nor shows that a normal client would see the same gap at the selected workload CL. Measurement isolation can prevent a lagging predecessor from arriving, while the wait before probing may hide an earlier transient gap. A PASS therefore cannot establish a general ordering guarantee. Recovery verification checks membership and connection reachability, not convergence of every key. The observed exceptions make that distinction concrete.

4.4 Conclusion

In these experiments, ONE allowed operations on both sides of a partition and exposed stale reads and local dependency gaps. QUORUM and ALL often prevented the required operation from completing instead. The connected-workload exceptions show why acknowledgement counts, per-replica visibility and successful client reads must be examined separately. The 316 histories support these specific observations; their counts are not estimates of general failure probabilities.

5 Reproduction and submitted evidence

The submission contains the report, editable source, code, tests and original records. Verify the saved results without running Cassandra:

```
python3 scripts/summarize_completed.py
```

The archive is `results/final-316-20260924/`. The offline check regenerates `audit.json` and `matrix-summary.json`. Each case preserves its history, environment, controls and completion record. The selection manifest records original hashes; `provenance/` contains both executed source snapshots and plans. The separate preparation-failure folder retains both interrupted executions. The original 1,080-case plans document intent, not completed counts.

To run new measurements, use a dedicated Docker daemon with about four CPU cores and 8 GB RAM, then run:

```
docker compose build cassandra1 runner
docker compose up -d --wait cassandra1 cassandra2 cassandra3
docker compose run --rm --no-deps runner \
python scripts/reproduce_independent.py --cases 316
```

Use `docker-compose` if Compose is installed separately; `--cases 36` runs one full block. Fresh runs use new keys and an output directory while following the same seeded case order. The portable runner applies phase 2's preparation policy throughout. It reproduces the workload, fault and probe logic, not the historical interruptions or exact outcomes. The two executed versions remain in the source snapshots.

The runner controls the three lab containers through the dedicated Docker socket. Run one suite at a time. Afterwards use `docker compose stop`; on the recorded Mac, also use `colima stop dsa5208`. The README provides installation and Overleaf instructions.

References and AI usage

- [1] D. B. Terry et al. (1994). [Session Guarantees for Weakly Consistent Replicated Data](#). Proceedings of PDIS. Accessed 24 September 2026.
- [2] Apache Cassandra 5.0 documentation. [Dynamo: replication and tunable consistency](#). Accessed 24 September 2026.
- [3] Apache Cassandra 5.0 documentation. [Read repair](#). Accessed 24 September 2026.
- [4] DSA5208 Canvas. [Project 1: Consistency Models in Distributed Databases](#). Assignment requirements supplied by the group; submission deadline 27 September 2026.
- [5] Project source code. [jiasiqi312/dsa5208](#).
- [6] The project idea and experimental approach came from our group members. OpenAI Codex assisted with code implementation and debugging, experiment execution and result checking, and report preparation and typesetting.